

STACKSPRINT | CEN3031

Repository Setup and Project Proposal Plan

CEN3031 - Leetcode Productivity App

September 15, 2026

Team: Santiago Ramirez, Matias Camaran, and Hussain.

Proposal status: Project name and Santiago's frontend role are confirmed; confirm remaining roles and details as estimates before submission.

1. Problem and proposed solution

A student practicing for technical interviews may practice sporadically, find it difficult to compare achievement to peers, and track their activity with disjointed tools. StackSprint will combine friendly competition, visible progress, and social discussion of solutions to encourage a daily coding-practice habit. While our users may maintain multiple activity sources their desired goal is consistent improvement towards passing technical interviews.

The solution extends the students current repository: Weekly rankings show progress among friends, problems can be discussed after completion, and both practice activity and project development are visible on a dashboard. Solve similar to Countdown. The intended users are students and other developers preparing for technical interviews.

Challenge alignment: Interpretation matches coding skills and visibility. Must confirm instructor supplied challenge statement matches before submitting.

2. Project vision

StackSprint brings teammates together to build consistent coding skills before technical interviews.

For students and early-career developers who need motivation and a clear view of their technical interview preparation, StackSprint is a social coding- practice companion that brings weekly competition, progress tracking, and discussions unlocked by verified problem completion. Unlike learning in isolation while checking separate activity trackers StackSprints weekly leaderboard and dashboard gamify repetitions into one cohesive learning journey.

Formatting was adjusted for readability. Check vision against the provided template in the text book before submission.

3. Team roles and collaboration

Member	Primary responsibility
Santiago Ramirez	Handles frontend design details, React components, accessibility, dashboard and leaderboard pages, and integration with the backend API.
Matias Camaran	Proposed Project Coordinator and Backend Lead. Track defined scope, task progress, API contract, database model, and authentication. Leads the development & release process.
Hussain	Works on data-source feasibility & Connector interfaces, activity integrations, automated testing, end-to-end integration checks, and monitoring risks.

Ideally, every teammate works on every feature but they own quality in different layers. All three members will review each others pull requests, write code, update assigned tasks, and keep team members aware of decisions. Ownership ensures accountability without risking the project if someone is unavailable.

4. Scope and success criteria

Feature	Minimum deliverable and acceptance criteria
Profiles and groups	User can sign in account and join a collaborative practice group. An authorized group is returned from server upon visiting the dashboard/profile page.
Weekly leaderboard	Rankings of group members by verified problems solved that week. Design a consistent UTC boundary for the week. Repeated submissions for the same problem do not increase the count; identical scores can have the same rank.
Unlockable discussions	Posts can be read and written if signed in and discussion is unlocked by account problem completion. Members cannot read or write locked discussions until completion has been verified. Includes server-side API request checks. Failure cases are covered by automated tests.
Activity dashboard	View number of problems solved total and per-week history with timestamps from connected accounts. Github events and local IDE activity each have separate metrics from problem-solving scores to prevent misleading or gamed rankings. Missing, loading, empty, and failed-sync states.
External integrations	Begin with research into automated verification with Leetcode. Add a Connector for Github activity after core UX is functional; consider WakaTime integration a bonus. Improve verification mocks to ensure simulated functionality is never confused with the real product during demo

Both positive and negative test cases are covered by automated tests. Connection failures are handled clearly and designed for retry. Clear marker indicates which sources are demo fixtures; never tell a user their demo credentials successfully connected to paid services.

Out of scope: “running” untrusted code submitted by users. Presenting Leetcode problem descriptions. Building an online judge platform. Native iOS or Android app development. Competing with similar global platforms publically. Initial release will focus on private groups of practicing students and professionals.

Definition of done: All acceptance criteria are met and confirmed by a teammate. All tests pass. Pull request is reviewed by another teammate and documentation is updated. Approved pull request is merged. Demonstration covers account sign-in, leaderboard score calculation, attempts to view locked/unlocked discussions, and error states on dashboard.

5. Proposed programming languages and technologies

Layer	Selection and rationale
Frontend	TypeScript, React HTML/CSS., Javascript when needed. Choose tooling that allows re-use between dashboard, rankings, and discussion boards.
Backend Software	Node. js with TypeScript. Language will be shared with frontend; they communicate through API. Architect to be understandable by entire team.
Data	PostgreSQL Storage is required for users, groups, practice problems, verified Leetcode submissions, weekly scores, and restricted discussion posts. Constraints prevent duplicate completion records; verification checks whether a problem was actually completed.
Testing and delivery	Vitest + Playwright when project scaffold exists. Delivery will use GitHub actions to trigger build and selected tests when code is committed. Git & GitHub Version control and Pull Request review. Public repo and GitHub Projects for task tracking.
Integration approach	Connectors will be built as independent abstractions over a source. Tokens should never be committed to Git. Leverage Ignored ENV files and a committed example file with placeholders.

6. Development and configuration management

Repository: <https://github.com/Santi852/Cen3031-Proj-1>

Main is used as the git-integration branch and substitutes Master Branch from the assignment sheet. Teammates should create short-lived feature/fix/docs branches when contributing changes and open Pull Requests to merge into main. During merge review at least one other teammate must approve the change.

Control	Team practice
Change workflow	Create a short-lived branch, make a focused change, and open a pull request. Obtain approval from another teammate before merging into main.
Branch protection	Branch Protection Enabled. Main is confirmed protected. Screenshot shows required pull requests, one approval and dismissal of stale approvals. Repository owner needs to verify detailed saved settings since account does not have permission to view them.
Environment consistency	Documentation of setup commands added when project scaffold is created. Migration and lockfiles committed to ensure consistency. Secrets remain out of Git; include safe environment-variable example. Review dependency changes and schema migrations via pull requests.
Review and recovery	Review comments before merging. Newly appended code to an approved pull request should be reviewed again. Use revert on a bad merge when possible. Use tags to mark agreed course milestones.
Quality checks	Manual checks will be used for planning documents. Use automated checks when application code exists. Ask repository owner to enforce passing checks before merging.
Task management	Columns are Backlog> Ready> In Progress> In Review> Done on GitHub project. Ownership is defined on each task and acceptance criteria is defined on each issue. Task cannot be moved to Done unless its work is confirmed to be complete.

7. Delivery plan

Stage	Deliverable	Lead
1 - Definition	Confirm requirements, vision, high-level roles & stack selections, integration feasibility, and backlog. Project Proposal	Completed by: Matias; Review Team
2 - Foundation	Application scaffold created, database schema designed, sign-in implemented with group model, and testable/test suite workflow.	Santiago: frontend; Matias: backend
3 - Core workflows	Leaderboard, problem completion integration, restricted discussions, and functional dashboard.	All
4 - Integration and release	Connector Interfaces, failsafe error handling, end-to-end testing, delivery demo, and project documentation.	Hussain; Review Team

These stages describe task ordering, not course milestones. Deadlines will be assigned using the course calendar. Teammates will review progress daily and bring blockers to the team within 24 hours. Matias is responsible for coordinating priority changes.

8. Risk management process

Identify: Review risk factors during project kickoff. Re-evaluate after any changes to scope. Risks should be recorded with an owner and an actional warning sign.

Assess and prioritize: Estimate likelihood and impact from 1(Low) - 3(High). Multiply values to determine Relative Priority. 1 - 2 Low Priority 3 - 4 Medium Priority 6 - 9 High Priority Scores are subjective and should only be used for ordering; they do not represent probabilities.

Respond: focus efforts on high-priority risks. Avoid risk by not doing work that will not be “done-right” within course limitations. Prepare for risk by defining behavior when the risk occurs before enabling the associated functionality. Common risks can be mitigated by allocating time to investigate during kickoff.

Monitor: Monitor risks during the regularly scheduled check-ins. Owners should provide current score, mitigation progress, and if the warning sign has been triggered. If triggered, create or update an issue, complete the contingency plan, and reassess scope with the team. Risks affecting course deliverables are escalated to the instructor via Matias.

Risk register: Risks specific to feasibility and project delivery.

R1: LeetCode Data is unavailable or changing.

Impact: 3 / High. **Likelihood:** 3 / High. **Priority:** 9 / High. **Owner:** Hussain. **Initial status:** Open.

Trigger: Research cannot uncover a method to confirm problem solves. Changes to source data. Server down or access revoked

Mitigation: Assess risks before starting features. Connectors should isolate dependency. Normalize incoming records. Record timestamps, be aware of rate-limits

Contingency: Use demonstration fixtures that are clearly labeled; seek instructor approval if a scope change reduces features or requires an alternative verification process. Discussion posts will not be unlocked based on self-reported problem solves.

R2: Project scope will exceed available time

Impact: 3 / High. **Likelihood:** 3 / High. **Priority:** 9 / High. **Owner:** Matias. **Initial status:** Open.

Trigger: Functionality is over a planned checkpoint or its relative effort exceeds the remaining capacity. Adjusted scope is approved and feature still doesn't meet checkpoint. Demonstrate or Shipping label falls of the projected timeline

Mitigation: Verify understand the core flows and confirm acceptance criteria. Core features have been identified. Stretch goals are outside of the minimum product.

Contingency: Strip out recognized Stretch goals, simplify workflow, remove unused UI. Ship something that works. Seriously.

R3: Unavailability of teammate or skills become blockers.

Impact: 2 / Medium. **Likelihood:** 3 / High. **Priority:** 6 / High. **Owner:** Matias. **Initial status:** Open.

Trigger: Risk has been unresolved for 1 day or an owner misses a project checkpoint. No response from a teammate for 24 hours. Blocked by risk being investigated

Mitigation: Documentation makes setup repeatable by anyone. Pair on risky features. Keep changes smaller than a teammates review.

Contingency: Assign another owner. Pair with teammate to resolve or reprioritize if capacity has dropped.

R4: Merge conflicts & inconsistent environment between teammates

Impact: 2 / Medium. **Likelihood:** 2 / Medium. **Priority:** 4 / Medium. **Owner:** Santiago. **Initial status:** Open.

Trigger: Branches have significantly diverged or a teammate cannot reproduce the same build/setup behavior. Communication gap is present between teammates. Merge request takes longer than expected to resolve

Mitigation: Use short-lived branches, Require PR review for changes. Document setup process, lock files, and agree on runtime versions

Contingency: Coordinate with owner to resolve conflicts. Consider rollback if merge broke the project build for another teammate. Fix setup instructions to prevent future regressions.

Risk register: Risks associated with sensitive data and presentation quality.

R5: Account or authorization data is exposed

Impact: 3 / High. **Likelihood:** 2 / Medium. **Priority:** 6 / High. **Owner:** Matias. **Initial status:** Open.

Trigger: Completion or access-control test fails or exposed credential appears in code/logs. Real tokens found in committed example files or code

Mitigation: Check authorization on server for grouped/discussion data. Do not expose local secrets. Audit config example. Review application logs.

Contingency: Disable affected functionality. Revoke exposed creds. Repair Rules & test before restoring affected functionality.

R6: Users are ranked incorrectly or their activity is duplicated

Impact: 2 / Medium. **Likelihood:** 2 / Medium. **Priority:** 4 / Medium. **Owner:** Hussain. **Initial status:** Open.

Trigger: Acceptance test for ties failed or verification tests don't meet defined check. Trigger occurs during final review. Scoring algorithm is inconsistent. Week boundary tests fail. Retries are counted multiple times

Mitigation: Clearly define start and end of a week. Use constraints to prevent duplicate scores. Document and test assumptions about ties, retries, etc.

Contingency: Temporarily hide ranks while logs are investigated. Adjust ranks based on verified completion records. Display correction message/log entry to users. Clarify source of risk and resolution steps.

R7: 3rd party service will become unreachable or enforce rate-limits

Impact: 2 / Medium. **Likelihood:** 2 / Medium. **Priority:** 4 / Medium. **Owner:** Hussain. **Initial status:** Open.

Trigger: Connector returned HTTP failure code(s) or known rate-limit message repeatedly. Downstream service impacts release functionality due to intermittent or nonexistent connectivity. Rate limits do not permit the planned refresh frequency.

Mitigation: Cache retrieved data where possible Show last-sync time. Implement automatic retries with limits. Build workflows that don't rely on real-time success.

Contingency: Notify users the data is stale. Pause retries until the service recovers or the rate-limit window expires, then resume manually or via a scheduled sync task. Rework proposed solutions to reduce risk likelihood.

R8: Defects are discovered late in the cycle impacts our critical flows

Impact: 2 / Medium. **Likelihood:** 2 / Medium. **Priority:** 4 / Medium. **Owner:** Santiago. **Initial status:** Open.

Trigger: Partner cannot complete demonstration or is blocked by ui lacking instructions/a control. Missing critical tests occur on approved software. Defect found during cx review. Defect discovered after soft deadline. Milestones are missed consistently

Mitigation: Review proposed wireframes as a team Validate entire flow including empty states, keyboard-accessibility, etc

Contingency: Fix release-blocking defects before shipping; defer minor cosmetic issues. Remove offending complexity from sprint. Simplify ui.

9. Submission tasks

Submit this file as PDF with screenshots of your project repository, configuration-management process documentation, branch-protection screen, and populated project board linked in your readme. Confirm remaining roles and stack before submitting. Confirm the challenge and vision template from your text book before submission. Review class calendar and assign task estimates before final submission.